

FASE 1: El Núcleo Lógico (Python Puro - Sin Interfaz)

Objetivo: Lograr que el algoritmo genere un turno válido en consola.

- **Paso 1.1:** Definir modelos de datos con Pydantic (Clases para Juez, Día, Turno, Regla).
- **Paso 1.2:** Crear scripts de ingestión (ETL) usando pandas para limpiar los 3 Excels de entrada y detectar errores (fechas, nombres duplicados).
- **Paso 1.3:** Implementar el **Solver** con Google OR-Tools.
 - Codificar variables booleanas (Juez J en Día D en Turno T).
 - Codificar "Hard Constraints" (Suma de turnos = 1, No repetición, etc.).
 - Codificar función objetivo (Maximizar equidad).
- **Paso 1.4:** Unit Testing del Solver. Crear casos de prueba (ej. "Caso imposible con todos los jueces enfermos") y verificar que el sistema falle elegantemente.

FASE 2: La Capa de Inteligencia (LangChain + OpenAI)

Objetivo: Que el agente pueda manejar el Solver.

- **Paso 2.1:** Configurar cliente OpenAI y definir el System Prompt.
- **Paso 2.2:** Crear las "Tools" de LangChain. Envolver las funciones de la Fase 1 (run_solver, read_excel) para que el LLM pueda llamarlas.
- **Paso 2.3:** Implementar el "Rule Override". Permitir que el LLM inyecte un JSON de configuración temporal al Solver basado en lenguaje natural (ej. "Cambia FPI a 3" -> {"FPI_CAPACITY": 3}).

FASE 3: Persistencia y Memoria (Backend)

Objetivo: Que el sistema recuerde.

- **Paso 3.1:** Diseñar esquema SQLite (Table: Cycles, Table: Assignments, Table: Users).
- **Paso 3.2:** Implementar ChromaDB. Crear función para guardar "texto libre" (notas del usuario) y recuperarlo por similitud semántica.
- **Paso 3.3:** Crear funciones de "Commit". Al aprobar un ciclo, guardar el JSON del resultado en SQLite y generar el .xlsx físico.

FASE 4: La Interfaz de Usuario (Frontend)

Objetivo: La experiencia de usuario.

- **Paso 4.1:** Configurar estructura base (ej. Electron con backend Flask/FastAPI).

- **Paso 4.2:** Crear componente de Chat (izquierda).
- **Paso 4.3:** Crear componente de Grid/Tabla (derecha) que renderice el JSON que manda el backend.
- **Paso 4.4:** Conectar botones de carga de archivos al script de Python.

FASE 5: Empaquetado y Despliegue

- **Paso 5.1:** Crear instalador con PyInstaller (para el backend) y Electron Builder.